

Sir Stokes der Zähige

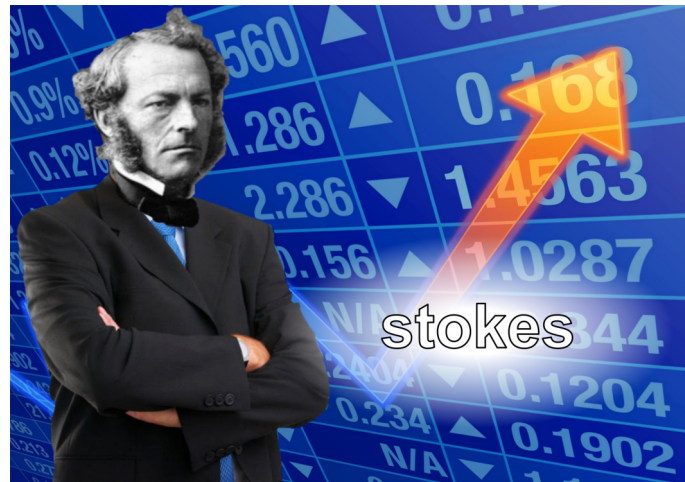


Abb. 0.1: MOTIVATION GO STOKES⁴

Auswertung Versuch 212 - Zähigkeit von Flüssigkeiten

Physikalisches Praktikum PAP 2.2

des Studienganges Physik

an der Universität Heidelberg

von

Benjamin Kleijn

12. August 2026

Versuchstermin 23.02.2026

Tutor:in Dimitrij Bathauer

Inhaltsverzeichnis

1. Einleitung	4
1.1. Motivation	4
1.2. Ziele des Versuches	4
1.3. Geräte	4
1.4. Theoretische und technische Grundlagen	5
1.4.1. Kugelfallviskosimeter nach Stokes	5
1.4.2. Laminare und turbulente Strömung	6
1.4.3. Viskositätsbestimmung nach Hagen-Poiseuille	6
2. Durchführung	7
2.1. Messverfahren	7
2.2. Messprotokoll	8
3. Auswertung	9
3.1. Stokes	10
3.2. Hagen-Poiseuille	13
3.3. Vergleich der Viskositäten	14
3.4. Differentialgleichung	15
4. Diskussion	18
4.1. Sinkgeschwindigkeiten	18
4.2. Hagen-Poiseuille	18
4.3. Vergleich der Versuchsteile	19
4.4. Fazit	19
Literaturquellen	20
Abbildungsquellen	20
A. Dichte von Polyethylenglykol	20
B. Memes	20
C. Code-Anhang	20

Abbildungsverzeichnis

0.1. <i>Meme</i> : Stonks ⁴	1
1.1. Versuchsaufbau/Geräte ¹	4
1.2. Viskosität anhand Flüssigkeitsschichten ⁵	5
1.3. laminare vs. turbulente Strömung ¹	6
2.1. <i>Meme</i> : PAP-Lebenskrise ⁶	7
2.2. Messdaten	8
3.4. Abbremszeiten bei hoher Anfangsgeschw.	16
3.5. Abbremszeiten bei realistischer Anfangsgeschw.	16
3.6. <i>Meme</i> : Erzählperspektive	17
A.1. Dichte von Polyethylenglykol ¹	37

Tabellenverzeichnis

3.1. korrigierte Geschwindigkeiten	12
3.2. Vergleich der berechneten Viskositäten	14

1. Einleitung

1.1. Motivation

Um wenigstens ein wenig Fluidodynamik in der Experimentalphysik zu behandeln, wird in diesem Versuch die Viskosität eines - interessanten - Öls (Polyethylenglykol kurz PEG) untersucht. Zusätzlich wird eine tolle Errungenschaft von Stokes untersucht: Stokes'sche Reibung. Endlich mal Reibung untersuchen :). Außerdem sind die Grundlagen des Modells eine schöne Wiederholung von Kräften.

1.2. Ziele des Versuches

Auf zwei unterschiedliche Weisen wird die Viskosität von PEG untersucht:

- durch Messen der Fallzeit über Stokes'sche Reibung
- durch Messen des Durchflusstroms in einem Kapillarviskosimeter nach dem Gesetz von Hagen-Poiseuille

Beim Kugelfallversuch wird zusätzlich der Übergang von laminarer zu turbulenter Umströmung der Kugel untersucht, um rudimentär die Gültigkeitsgrenze des Stokes'schen Gesetzes zu bestimmen. Die mit beiden Methoden erhaltenen Werte werden abschließend verglichen und auf Konsistenz unter Berücksichtigung der Messunsicherheiten geprüft.

1.3. Geräte

1. Messzylinder/-behälter, gefüllt mit Polyethylenglykol (PEG), aufgedruckter Messskala und Präzisionskapillare am unteren Ende
2. Probekugeln aus Polyacetal (ρ_K) mit verschiedenen Durchmessern ($(1.000 \pm 0.025$ bis $9.000 \pm 0.025)$ mm)
3. Thermometer
4. Pinzetten und Bechergläser
5. Stoppuhr



Abb. 1.1: Versuchsaufbau/Geräte¹

1.4. Theoretische und technische Grundlagen¹

Die innere Reibung einer Newtonschen Flüssigkeit zwischen einer stationären und einer bewegten Platte mit Abstand d ist proportional zur tangentialen Scherkraft F , zur Fläche A und zum Geschwindigkeitsgradienten $\frac{dv}{dd}$. Bei gleichförmiger Bewegung der oberen Platte mit Kraft F und Geschwindigkeit v ergibt sich, dass die Reibungskraft den selben Betrag hat. Diese Beziehung definiert die Viskosität η :

$$F_r = \eta A \frac{dv}{dd} \rightarrow [\eta] = \text{Pa s} \quad (1.1)$$

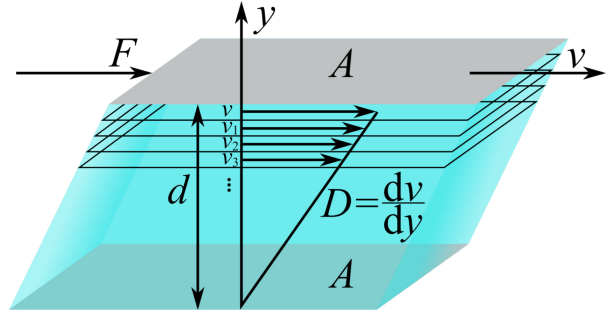


Abb. 1.2: schichtweises Bewegen der Flüssigkeit Richtung Kraft F ⁵

1.4.1. Kugelfallviskosimeter nach Stokes

Für eine Kugel mit Radius r , die sich mit konstanter (Sink-)Geschwindigkeit v durch eine (viskose) Flüssigkeit bewegt, gilt nach dem Stokes'sche Gesetz für die Reibungskraft:

$$F_R = 6\pi \eta r v \quad (1.2)$$

Dieses Gesetz gilt nur für unendlich ausgedehnte Flüssigkeiten, deswegen wird die Reibung für ein Rohr des Radius R mit einem Korrekturfaktor λ (Ladenburgsche Korrektur) angeglichen, die Näherung lautet

$$F_R = 6\pi \eta r v \lambda \quad \text{mit} \quad \lambda = 1 + 2.1 \frac{r}{R} \quad (1.3)$$

Für eine fallende Kugel mit Materialdichte ρ_k , Flüssigkeitsdichte ρ_F und Kugelvolumen $V_k = \frac{4}{3}\pi r^3$ stellt sich im stationären Zustand ein Kräftegleichgewicht zwischen Gewichtskraft, Auftrieb und Reibung ein:

$$\underbrace{\rho_K V_K g}_{F_g} - \underbrace{\rho_F V_k g}_{F_a} - \underbrace{6\pi \eta r \lambda v_s}_{F_r} = 0. \quad (1.4)$$

Durch umformen ergibt sich:

$$\eta = \frac{2}{9} g \frac{\rho_k - \rho_f}{v_s} r^2 \quad (1.5)$$

1.4.2. Laminare und turbulente Strömung

Man unterscheidet in der Fluidodynamik zwischen laminaren und turbulenten Strömungen, erstere sind eine Idealisierung, jedoch eine zum Beschreiben des Systems freundliche Näherung.

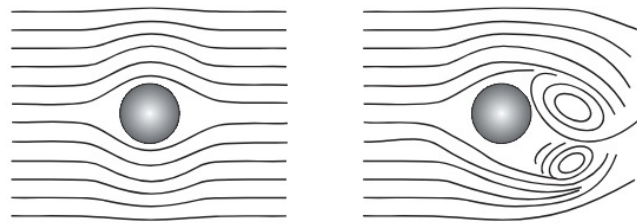


Abb. 1.3: Links: Laminare Strömung, keine Vermischung der Flüssigkeitsschichten.

Rechts: Turbulente Strömung bei hohen Geschwindigkeiten, es kommt zu Vermischung der Flüssigkeit¹

Das Verhalten der Strömung lässt sich mit der dimensionslose Reynoldszahl charakterisieren, welche das Verhältnis von Trägheits- zu Reibungskräften beschreibt. Für ein Strömungssystem ist Re definiert durch:

$$Re = \frac{\rho v L}{\eta}, \quad (1.6)$$

wobei L eine charakteristische Länge ist (für die Kugelbewegung $L = 2r$, für Rohrströmung $L = 2R$). Für Rohrströmungen liegt der Übergang zu Turbulenz typischerweise bei $Re \approx 2300$; für die Umströmung einer Kugel ist der kritische Bereich deutlich kleiner (experimentell $Re^{\text{krit}} \approx 1$).

1.4.3. Viskositätsbestimmung nach Hagen-Poiseuille

Zur Viskositätsbestimmung nach Hagen-Poiseuille wird nun eine Rohrströmung in der Kapillare (Radius R , Länge L) am unteren Zylinderende betrachtet. Denn durch die aus der Druckdifferenz $\Delta p := p_1 - p_2$ entstehende Kraft $F_p = \pi r^2 \Delta p$ fließt die Flüssigkeit aus der Kapillare hinaus. Dieser Kraft wirkt die Reibungskraft (Gleichung (1.1)) $F_r = 2\pi r L \eta \frac{dv}{dr}$ entgegen. Beachte: Die Kräfte wirken pro Flüssigkeits-Teilzylinder mit Radius $r < R$ und Mantelfläche $A = 2\pi r L$. Unter Annahme der Randbedingung $v(R) = 0$ und dem Vorliegen einer laminaren Strömung, wird sich durch den Gleichgewichtszustand zwischen F_r und F_p ein parabolisches Geschwindigkeitsprofil entwickeln.

$$\xrightarrow{F_p=F_r} \frac{dv}{dr} = \frac{\Delta p}{2\eta L} r \quad \xrightarrow[v(R)=0 \text{ und}]{\text{Integration über } r} v(r) = \frac{\Delta p}{4\eta L} (R^2 - r^2) \quad (1.7a)$$

$$\xrightarrow[\text{pro Teilzylinder}]{\text{Volumenstrom}} \frac{dV}{dt} = 2\pi r v(r) \quad \xrightarrow[\text{über } r]{\text{Integration}} \dot{V} = \frac{\pi \Delta p R^4}{8\eta L} \quad (1.7b)$$

Und so kann final die die Viskosität bestimmt werden:

$$\eta = \frac{\pi \Delta p R^4}{8L \dot{V}} \quad (1.8)$$

Pfeil mit
tikz zur
Formel
wäre wild
– Done :)

2. Durchführung

2.1. Messverfahren

Durch mittiges Eintauchen der Plastikkügelchen mit einer Pinzette im PEG-gefüllten Messzylinder wird der Sinkvorgang initiiert. Die Fallgeschwindigkeit wird offensichtlicherweise durch Messen einer Fallstrecke und zugehöriger Fallzeit erhoben - NoShitSherlock. Dies wird pro Durchmesser mit 5 Kugeln wiederholt, sodass pro Durchmesser eine Messreihe evaluiert werden kann.

Im zweiten Teil, nach Hagen-Poiseuille, wird, in einer äußerst weltbewegenderen Weise als in Versuchsteil 1, nun durch Messen der Durchflussmenge V (Becherglas mit Skala) pro Zeit t (Stoppuhr), die zur Auswertung erforderliche Größe des Durchflusstroms $\dot{V}$ bestimmt. Zusätzlich ist hier, nach Gleichung (1.8), die Bestimmung des an der Kapillare anliegenden Druckes notwendig, was mithilfe einer Höhenmessung des Fluids im Messzylinder erfolgt.

allpapliche Lebenskrise



Abb. 2.1: Hier war Platz auf der Seite.

Fourieroptik war der beste Versuch (rein nach Durchführung und Theorie)⁶

2.2. Messprotokoll

Versuch 212

Nr.1

Temperaturen [°C] 21.80 ΔT [°C] 0.5

		Zeiten [s]								
2*Radius [mm]	Fallstrecke [cm]	t ₁	t ₂	t ₃	t ₄	t ₅	$\bar{t}$ [s]	Δt [s]	v [m/s]	Δv [m/s]
delta_d=0.0025mm	$\Delta s=0.5\text{cm}$	$\Delta t=0.3\text{s}$					Stddeviation			
9	40.0	13.1	13.2	13.3	13.5	13.4	13.30	0.25	0.0301	0.0007
8	40.0	15.4	15.7	16.3	15.9	15.8	15.8	0.4	0.0253	0.0007
7	30.0	15.1	14.9	14.8	14.7	15.1	14.92	0.27	0.0201	0.0005
6	20.0	13.2	13.0	13.9	13.6	13.6	13.5	0.4	0.0148	0.0006
5	10.0	8.7	8.7	9.3	8.2	8.7	8.7	0.5	0.0115	0.0009
4	5.0	7.0	7.1	6.9	6.6	7.4	7.0	0.4	0.0071	0.0008
3	5.0	11.3	11.2	11.6	11.2	11.5	11.36	0.27	0.0044	0.0005
2	5.0	26.8	26.7	24.9	26.6	25.7	26.1	0.9	0.00192	0.00020
1	2.0	34.7	31.4	31.2	34.5	33.8	33.1	2	0.00060	0.00016

T₂ [°C] 22.0 ΔT [°C] 0.5

T₃ [°C] 22.4

Nr.2

Höhe h [ml]	Stoppzeit t [s]	
$\Delta h = 1\text{ml}$	$\Delta t = 0.3\text{s}$	t _(i+1) -t _i
5.0	173.0	171.1
10.0	344.2	170.2
15.0	514.4	175.4
20.0	689.8	172.7
25.0	862.5	171.3
30.0	1033.8	

Δh [ml] 1

R Rohr [mm] 75 Fehler unknown
 Anfangshöhe [mm] 544.0 $\pm 1.0\text{ ml}$
 Endhöhe [mm] 536.00 $\pm 1.0\text{ ml}$
 Mittelwerthöhe [mm] 540.0 0.7

Abb. 2.2: fig:Messprotokoll

3. Auswertung

Formeln der statistischen Analyse²

Varianzen: Für die statistische Auswertung von n Messwerten $\{x_i\}_{i=1}^n$ werden folgende Größen definiert:

$$\text{Arithmetisches Mittel} \quad \bar{x} = \text{Mean}(\{x_i\}_{i=1}^n) = \frac{1}{n} \sum_{i=1}^n x_i \quad (\text{S.1})$$

$$\text{Varianz} \quad \sigma^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2 \quad (\text{S.2})$$

$$\text{Standardabweichung} \quad \sigma = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2} \quad (\text{S.3})$$

$$\text{Fehler des Mittelwerts} \quad \Delta \bar{x} = \frac{\sigma}{\sqrt{n-1}} \quad (\text{S.4})$$

Fehlerfortpflanzung: Der Fehler einer Funktion $f(x_1, \dots, x_N)$, die von den Variablen $\{x_i\}_{i=1}^N$ abhängt, wird wie folgt bestimmt:

$$\text{Gauß'sche Fehlerfortpfl.} \quad \Delta f = \sqrt{\sum_{i=1}^N \left(\frac{\partial f}{\partial x_i} \Delta x_i \right)^2} \quad (\text{S.5})$$

$$\text{relativer Fehler von } f = \prod_{i=1}^N x_i^{\alpha_i} \quad \frac{\Delta f}{|f|} = \sqrt{\sum_{i=1}^N \left(\frac{\alpha_i \Delta x_i}{x_i} \right)^2} \quad (\text{S.6})$$

Ergebnissignifikanz: Resultate eines Experiments bzw. Berechnung a_{exp} und die entsprechenden Literaturwerte a_{lit} werden folgend verglichen:

$$\text{Absolute Abweichung} \quad A = |a_{\text{lit}} - a_{\text{exp}}|$$

$$\Delta A = \sqrt{(\Delta a_{\text{lit}})^2 + (\Delta a_{\text{exp}})^2} \quad (\text{S.7})$$

$$\text{Relative Abweichung} \quad R[\%] = \frac{A}{a_{\text{lit}}} \cdot 100 \quad (\text{S.8})$$

$$\text{Signifikanz} \quad S[\sigma] = \frac{A}{\Delta A} = \frac{|a_{\text{lit}} - a_{\text{exp}}|}{\sqrt{\Delta a_{\text{lit}}^2 + \Delta a_{\text{exp}}^2}} \quad (\text{S.9})$$

Fitgüte: Für einen Fit mit Funktionswerten $\{F_{a_1, \dots, a_m}(x_i)\}_{i=1}^n$ und Fitparametern $\{a_j\}_{j=1}^m$ an die experimentellen Messwerte $\{y_i\}_{i=1}^n$ mit Fehler $\{\Delta y_i\}_{i=1}^n$ kann die Güte des Fits mit diesen Größen analysiert werden:

$$\chi^2 \text{ Summe} \quad \chi^2 = \sum_{i=1}^n \left(\frac{F(x_i) - y_i}{\Delta y_i} \right)^2 \quad (\text{S.10})$$

$$\text{Freiheitsgrade} \quad f = n - m \quad (\text{S.11})$$

$$\text{normierte reduzierte } \chi^2 \text{ Summe} \quad \chi_{\text{red}}^2 = \frac{\chi^2}{f} \quad (\text{S.12})$$

$$\text{Fitwahrscheinlichkeit} \quad p = 1 - \text{chi2.cdf}(\chi^2, f) \quad (\text{S.13})$$

Hierbei wird das Modul `chi2` von `scipy.stats` benutzt.

3.1. Stokes

Plotten der Sinkgeschwindigkeiten

Als ersten Schritt rechnet man die 5 erfassten Messreihen von Sinkzeiten t und Sinkstrecke s mit Fehler in die Geschwindigkeit um. Hierfür wird zuerst aus den pro Messreihe 5 erfassten Zeiten der Mittelwert $\bar{t}$ (Gleichung (S.1)) berechnet. Da jedoch nur 5 Messwerte zur Verfügung stehen, wurde für $\Delta \bar{t}$ nicht der Fehler des Mittelwerts genommen, sondern nur die klassische Standardabweichung σ . Zusätzlich wurde quadratisch der Reaktionszeitfehler $\Delta t = 0.3 \text{ s}$ addiert (entsteht aus $\sqrt{2} \cdot 0.2 \text{ s}$, den Reaktionszeiten beim Starten und Stoppen).

$$\Delta \bar{t} = \sqrt{\sigma(t_i)^2 + (0.3 \text{ s})^2} \quad (\text{3.1})$$

Mithilfe des bekannten Fehlers $\Delta s = 0.005 \text{ cm}$ kann nun die Geschwindigkeit mit Fehler berechnet werden:

$$v = \frac{s}{t} \quad \Delta v = \sqrt{\left(\frac{\Delta s}{t} \right)^2 + \left(\frac{s \Delta t}{t^2} \right)^2} \quad (\text{3.2})$$

Zusätzlich werden mit der Ladenburgschen Korrektur $\lambda v = (1 + 2.1 \frac{r}{R})v$ versehene Geschwindigkeiten geplottet. Es war kein Fehler des Rohrradiuses R angegeben. Dieser wird deswegen als exakt angenommen. Fehler der korrigierten Geschwindigkeit ergibt sich zu:

$$\Delta(\lambda v) = \sqrt{\left(\Delta v \left(1 + \frac{2.1r}{R} \right) \right)^2 + \left(\Delta r \frac{2.1v}{R} \right)^2} \quad (\text{3.3})$$

Die Änderung von Δv ist nur minimal. An diese neuen Werte wurde eine Ursprungsgerade gefittet (siehe Abb. 3.1). Mithilfe deren Steigung a kann nun die Viskosität η berechnet werden. Genutzt wurde hier die im Skript angegebene Dichte der Kügelchen: $\rho_K = (1.385 \pm 0.015) \cdot$

10^3 kg m^{-3} sowie die zur Raumtemperatur $T = (21.9 \pm 0.5) ^\circ\text{C}$ passende PEG-Dichte (abgelesen in Abb. A.1) $\rho_F = (1.1478 \pm 0.0005) \cdot 10^3 \text{ kg m}^{-3}$.

$$\eta = \frac{2g(\rho_K - \rho_F)}{9a} \quad (3.4)$$

$$\Delta\eta = \sqrt{\left(\frac{\Delta g 2(-\rho_F + \rho_K)}{9a}\right)^2 + \left(-\frac{2\Delta\rho_F g}{9a}\right)^2 + \left(\frac{2\Delta\rho_K g}{9a}\right)^2 + \left(-\frac{\Delta a g 2(-\rho_F + \rho_K)}{9a^2}\right)^2} \quad (3.5)$$

$$\eta^{\text{Stokes}} = (0.268 \pm 0.004) \text{ Pa s}$$

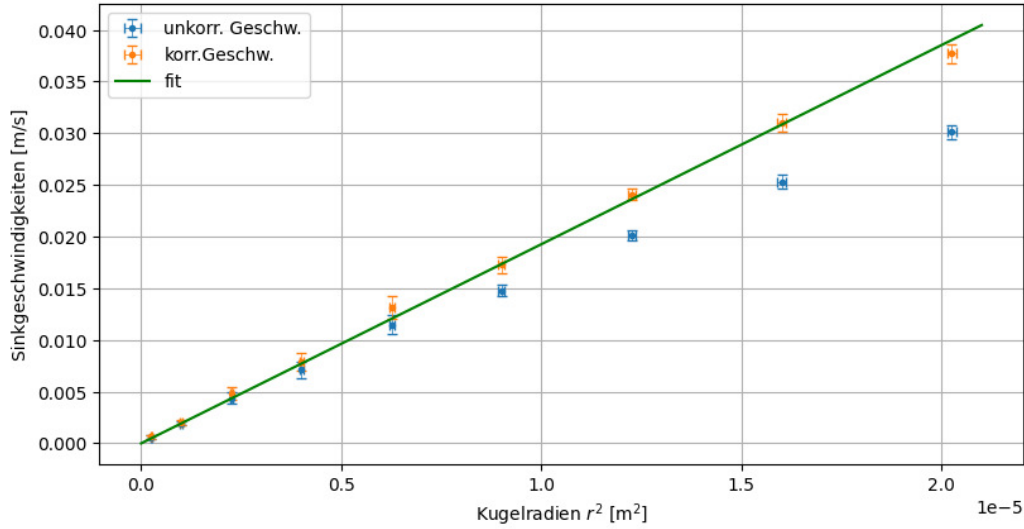


Abb. 3.1: uncorr. und korr. Sinkgeschwindigkeiten der K gelchen mit Fitgerade

Berechnen der v_{lam} Sinkgeschwindigkeiten

Mithilfe η^{Stokes} aus dem Fitergebnis (Steigung a) k nnen nun die zu den Kugelradien geh renden, theoretischen Sinkgeschwindigkeiten v_{lam} berechnet werden:

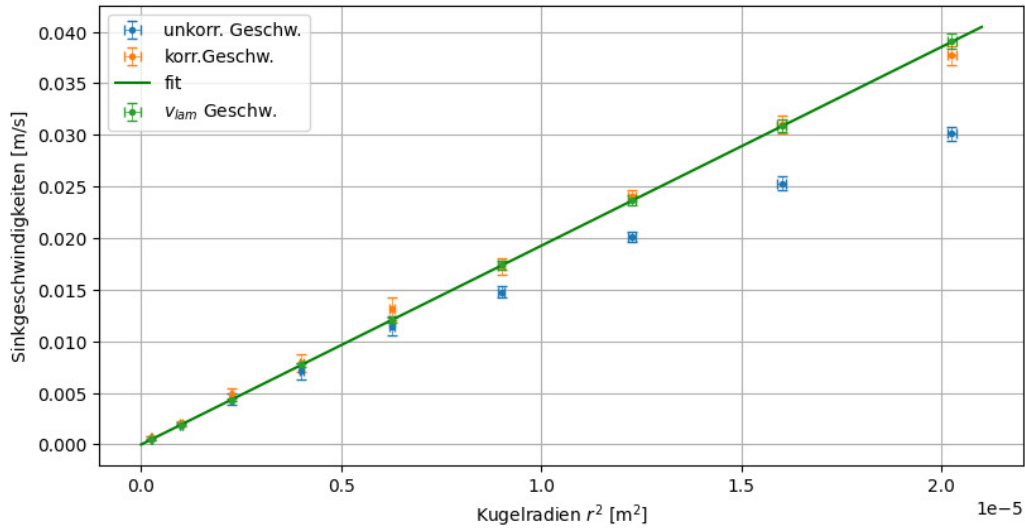
$$v_{lam} = \frac{2gr^2(-\rho_F + \rho_K)}{9\eta} \quad (3.6)$$

$$\frac{\Delta v_{lam}}{v_{lam}} = \sqrt{\frac{\Delta\eta^2}{\eta^2} + \frac{\Delta g^2}{g^2} + \frac{4\Delta r^2}{r^2} + \frac{\Delta\rho_F^2}{(\rho_F - \rho_K)^2} + \frac{\Delta\rho_K^2}{(\rho_F - \rho_K)^2}} \quad (3.7)$$

Tabelle 3.1: korrigierte Geschwindigkeiten

$r[\text{mm}]$	$v[\text{m s}^{-1}]$	$v_{lam}[\text{m s}^{-1}]$
4.5	0.0377 ± 0.0009	0.0390 ± 0.0007
4.0	0.0310 ± 0.0009	0.0309 ± 0.0006
3.5	0.0240 ± 0.0006	0.0236 ± 0.0005
3.0	0.0173 ± 0.0007	0.0174 ± 0.0004
2.5	0.0131 ± 0.0001	0.01204 ± 0.00024
2.0	0.0079 ± 0.0009	0.00771 ± 0.00016
1.5	0.0048 ± 0.0005	0.00434 ± 0.00011
1.0	0.00203 ± 0.00021	0.00193 ± 0.00006
0.5	0.00062 ± 0.00016	0.00049 ± 0.00003

In der folgenden Grafik sieht man dann nochmal schön, dass die Werte v_{lam} perfekt auf der Ausgleichsgeraden liegen, da sie mithilfe deren Steigung berechnet wurden.

**Abb. 3.2:** Eintragen der unzähligen Werte

Reynoldszahl

$$Re = \frac{2r\rho_F v}{\eta} \quad (3.8)$$

$$\frac{\Delta Re}{Re} = \sqrt{\frac{\Delta_{\eta}^2}{\eta^2} + \frac{\Delta_r^2}{r^2} + \frac{\Delta_{\rho_F}^2}{\rho_F^2} + \frac{\Delta_v^2}{v^2}} \quad (3.9)$$

Das Verhältnis $\frac{v}{v_{lam}}$ wird nun gegen $\log(Re)$ aufgetragen, die Reynoldszahlen Re wurden durch

die obigen Formeln (nach Gleichung (1.6)) berechnet. Der Fehler des Geschwindigkeitsverhältnisses ergibt sich mit:

$$\Delta \left(\frac{v}{v_{lam}} \right) = \sqrt{\left(\frac{\Delta v}{v_{lam}} \right)^2 + \left(\frac{v \Delta v_{lam}}{v_{lam}^2} \right)^2} \quad (3.10)$$

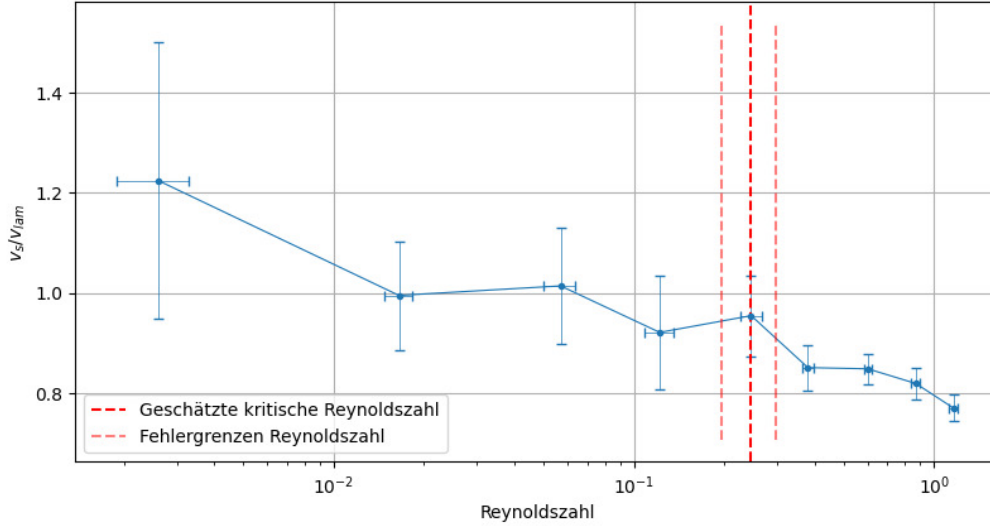


Abb. 3.3: verzweifelter Extrahieren der Grenze von laminarer zu turbulenter Strömung

Als kritische Reynoldszahl wird aus Abb. 3.3 abgeschätzt:

$$Re^{krit} = 0.246 \pm 0.050$$

3.2. Hagen-Poiseuille

Mithilfe des Kapillarrohrs und einem dieses durchfließenden Volumenstroms kann die Viskosität auf die Weise von Hagen-Poiseuille berechnet werden, siehe Gleichung (1.8). Hier wird zuerst die Druckdifferenz mit dem hydrostatischen Druck p ausgedrückt (PFFFFTTT):

$$p = \rho_F g h \quad \Delta p = p \cdot \sqrt{\left(\frac{\Delta h}{h} \right)^2 + \left(\frac{\Delta \rho_F}{\rho_F} \right)^2 + \left(\frac{\Delta g}{g} \right)^2} \quad (3.11)$$

Dieser berechnet sich mit $g = (9.809\,84 \pm 0.000\,02) \text{ m s}^{-2}$, $h = 0.540 \text{ m}$ zu $p = (6079.0 \pm 9.0) \text{ Pa}$. Da sowohl das große Rohr und die Kapillare offen sind, liegt an beiden der Atmosphärendruck an, welcher sich in der Druckdifferenz rauskürzt. Die Messdaten $V(t)$ aus dem Messprotokoll lassen sich nun durch Berechnen des Mittelwerts von $\frac{dV}{dt} = \dot{V}$ (jedoch Auslassen des Werts $V = 5 \text{ mL}$) in Gleichung (1.8) einsetzen. Die Werte $R = (1.50 \pm 0.01) \text{ mm}$ und $L = (100.0 \pm 0.5) \text{ mm}$ sind

im Skript angegeben. Nach Berechnen des Fehlers ergibt sich:

$$\Delta\eta = \eta \sqrt{\frac{\Delta_L^2}{L^2} + \frac{\Delta\dot{V}^2}{\dot{V}^2} + \frac{\Delta_p^2}{p^2} + \frac{16\Delta_r^2}{r^2}} \quad (3.12)$$

$$\eta^{\text{Hagen-Poiseuille}} = (0.261 \pm 0.017) \text{ Pa s}$$

Reynoldszahl: Wie schon bei Stokes geschehen, wird nun auch die Reynoldszahl mit Gleichung (3.9) berechnet. Für v wird die mittlere Geschwindigkeit $\bar{v} = \frac{\dot{V}}{\pi R^2}$ eingesetzt, r ist der Durchmesser R der Kapillare. Somit erhält man:

$$Re = \frac{2\rho_F \dot{V}}{\pi\eta R} \quad \Delta Re = Re \cdot \sqrt{\frac{\Delta_{Lkap}^2}{L_{kap}^2} + \frac{\Delta_{dVdt}^2}{dVdt^2} + \frac{\Delta_p^2}{p^2} + \frac{16\Delta_{rkap}^2}{r_{kap}^2}} \quad (3.13)$$

$$Re^{\text{Hagen-Poiseuille}} = 0.109 \pm 0.011$$

Damit liegt die Reynoldszahl für die vorliegenden Charakteristika (PEG-Dichte ρ_F , Rohrdurchmesser $2r$ und Strömungsgeschwindigkeit v) deutlich unter der kritischen Reynoldszahl ($Re_{kr} = 2300$ einer Rohrströmung) und wir können folglich tatsächlich von laminarer Strömung ausgehen, die Verwendung des Hagen-Poiseuille-Gesetzes ist also gerechtfertigt.

3.3. Vergleich der Viskositäten

Die Ergebnisse der zwei Versuchsmethoden werden hier verglichen:

Tabelle 3.2: Vergleich der berechneten Viskositäten

$\eta^{\text{Stokes}} [\text{Pa s}]$	$\eta^{\text{Hag.-Pois.}} [\text{Pa s}]$	$S[\sigma]$
0.268 ± 0.004	0.261 ± 0.017	0.4

3.4. Differentialgleichung

Aus dem Kräftegleichgewicht in Gleichung (1.4) lässt sich bestimmen, wie sich die Geschwindigkeit $v(t)$ des Teilchens nach Eintauchen ändert. Dafür stellt man folgende Differentialgleichung auf:

$$\underbrace{m\dot{v}}_{F_{res.}} = \underbrace{\rho_K V_K g}_{F_g} - \underbrace{\rho_F V_k g}_{F_a} - \underbrace{6\pi\eta r \lambda v_s}_{F_r} \quad (3.14)$$

$$\dot{v} = \underbrace{\frac{4\pi r^3(\rho_K - \rho_F)}{3m}}_C - \underbrace{\frac{6\pi\eta r}{m}}_D v = C - Dv$$

Diese inhomogene DGL 1.Ordnung lässt sich mithilfe von Variation der Konstanten lösen: Die Lösung des homogenen Teils ist $v_h = K(t)e^{-Dt}$, dieser Ansatz wird nun in die inhomogene DGL eingesetzt:

$$\begin{aligned} \frac{d}{dt} (K(t)e^{-Dt}) &= \dot{K}(t)e^{-Dt} - K(t)De^{-Dt} = C - DK(t)e^{-Dt} \\ \dot{K}(t)e^{-Dt} &= C \\ K(t) &= \int_0^t Ce^{Dt'} dt' = \frac{C}{D}e^{Dt} - \frac{C}{D} + v_0 \end{aligned} \quad (3.15)$$

v_0 entsteht aus den Integrationskonstanten.

Setzt man diese Lösung für $K(t)$ nun wieder in die homogene Lösung ein, erhält man mit der Sinkgeschwindigkeit $v_s = \frac{C}{D}$ das finale Ergebnis

$$v(t) = \frac{C}{D} - \frac{C}{D}e^{-Dt} + v_0e^{-Dt} = v_s(1 - e^{-Dt}) + v_0e^{-Dt}$$

Da es hier nur darum geht, ein Gefühl für die Größenordnungen zu bekommen, wird hier die Viskosität als $\eta = 0.2 \text{ Pas}$ angenommen. Man kann im Code verschiedene Startgeschwindigkeiten einstellen, und die Geschwindigkeit $v(t)$ plotten lassen. Diese wird natürlich stark vom radiusabhängigen D -Wert beeinflusst, dieser skaliert mit r^{-2} , somit sieht man in Abb. 3.4, dass der Geschwindigkeitsabfalls für kleinere Radien zügiger von statten geht. Im Plot werden 3 berechnete Dinge angezeigt:

$$\begin{aligned} v_{\text{beschleuniger}}(t) &= v_0 \cdot e^{-Dt} \text{} \\ v(t) &= v_s(1 - e^{-Dt}) + v_0 \cdot e^{-Dt} \\ t_{\text{schnitt}} &= \frac{\ln(v_s/v_0)}{-a} \end{aligned}$$

Die Abbremszeit t_{schnitt} wird berechnet, indem wie bei den Zusatzaufgaben³ vorgeschlagen wird, die relevante Größenordnung als v_s festgelegt wird, auf die $v_{\text{beschl.}}(t)$ absinken soll.

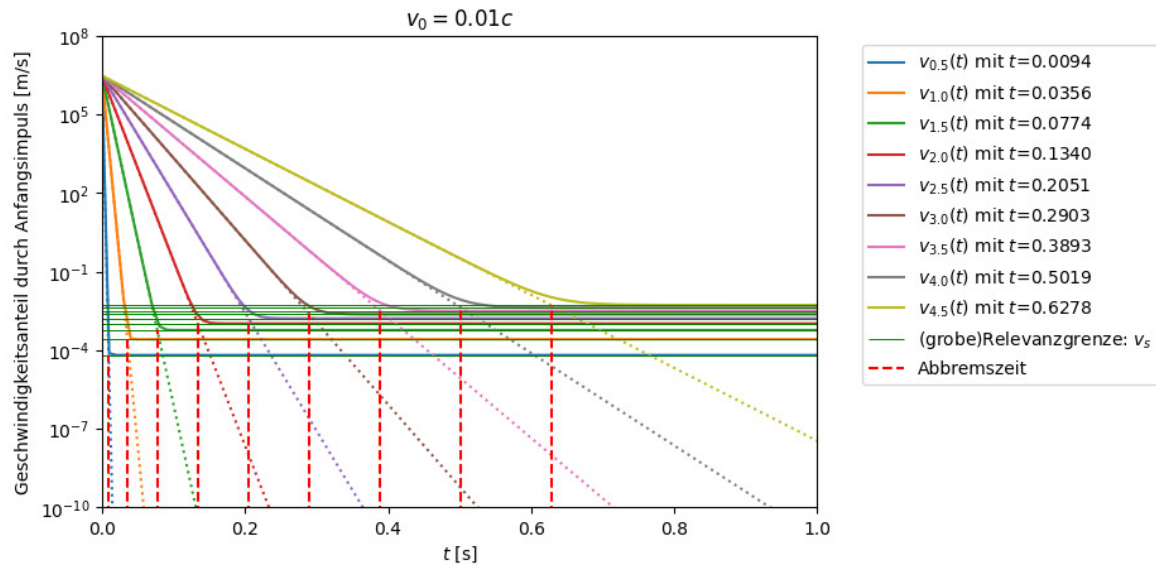


Abb. 3.4: Abbremszeiten für die Kugeln bei Startgeschwindigkeit

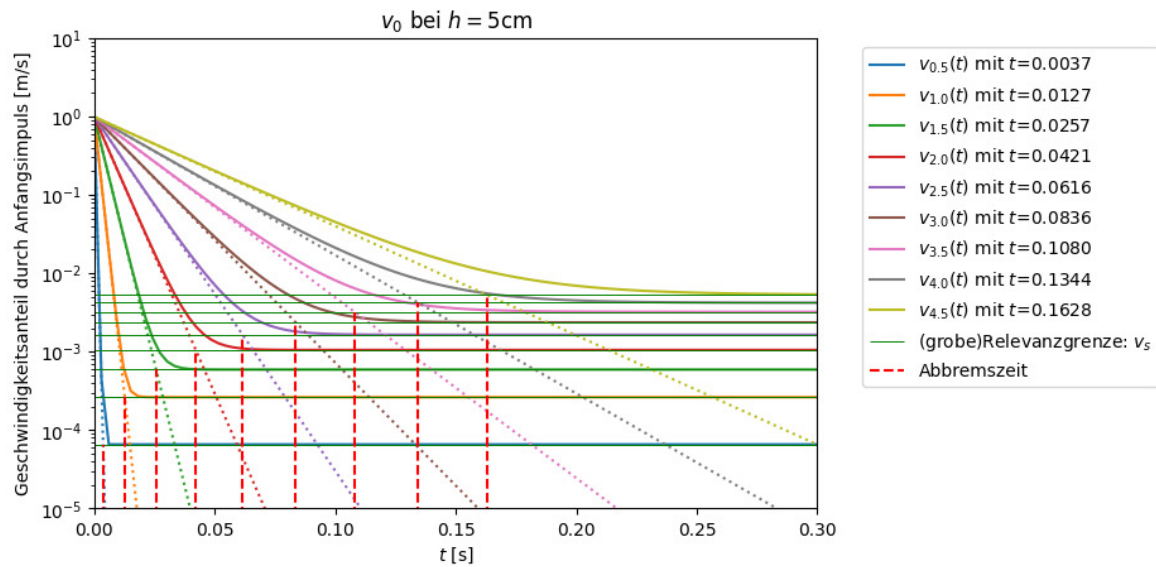


Abb. 3.5: Abbremszeiten für die Kugeln bei Startgeschwindigkeit die zweite

Man beobachtet, dass die Zeiten natürlich gemäß dem Logarithmus mit sinkender Startgeschwindigkeit immer kleiner werden, für unsere Messergebnisse spielt der Abbremsvorgang also keine signifikante Rolle.

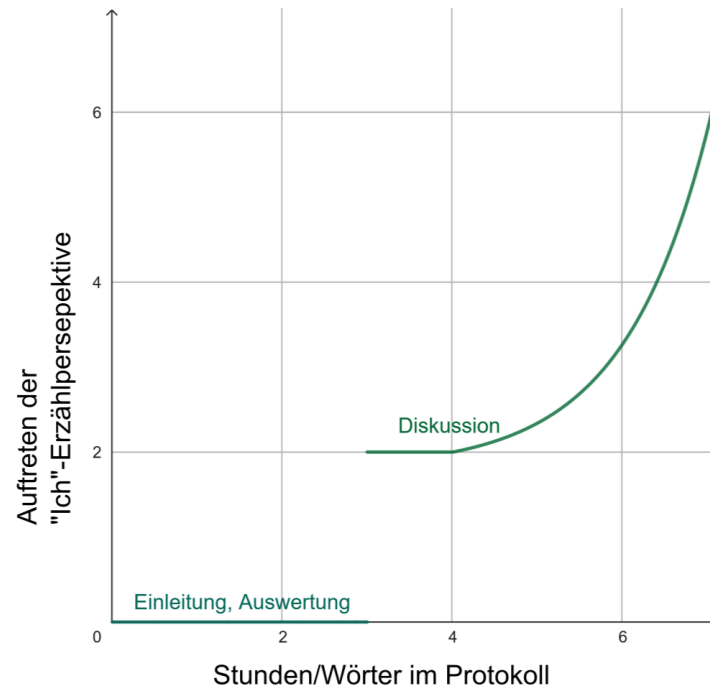


Abb. 3.6: Langsam reicht es auch mit Memes. Je länger ich wach bin, desto ineffizienter wird alles

4. Diskussion

4.1. Sinkgeschwindigkeiten

Zeitmessfehler Den relativen Fehler der systematischen Zeitmessung kann man natürlich durch eine Verlängerung der Messzeit erreichen. Erhöht man die Zahl N der Messpunkte, so kann auch der Mittelwertsfehler (statistischer Fehler) genutzt werden, welcher durch höheres N nach unten gedrückt wird.

linearer Fit Nicht ganz klar war mir, anhand welcher korrigierten Werte der lineare Fit durchgeführt werden soll, denn in Abb. 3.1 sieht man, dass sowohl unkor. als auch korr. Geschwindigkeiten keinen Unterschied in ihrem linearen Verhalten FÜR ALLE Radien zeigen. Es ist kein Sprung zu sehen, anhand dessen man argumentieren könnte, die höheren Radien für den Fit einer laminaren Strömung auszuschließen.

Geschwindigkeitsverhältnis und Reynoldszahl In Abb. 3.3 sieht man jetzt den eigentlich vorher schon erwarteten Trend/Effekt, dass die real gemessene Sinkgeschwindigkeit für höhere Radien aufgrund turbulenterem Strömungsverhalten kleiner ist, als die nach dem laminaren Modell erwartete Geschwindigkeit v_{lam} . Dies sieht man daran, dass das Verhältnis $\frac{v}{v_{lam}}$ mit größerem Radius sinkt.

Der zu suchende Knick im Graphen war sogar aufzufinden, seine Fehlergrenzen sind aber nur rudimentär geeyeballed. Der Wert von $Re^{krit.} = 0.246 \pm 0.050$ liegt somit bei $1/4$ der im Skript¹ angegeben kritischen Reynoldszahl einer sinkenden Kugel.

4.2. Hagen-Poiseuille

Fehlerrechnung Der Grund, warum der Fehler von $\eta^{Hag.Pois.}$ deutlich größer ist, als bei der Stokes-Methode, ist, dass ich diesen auf eine sehr fragwürdige Weise berechnet habe:

$$\Delta \dot{V} = \sqrt{\sigma^2\left(\frac{V_i}{t_i}\right) + mean^2\left(\frac{V_i}{t_i} \cdot \sqrt{\frac{\Delta t_i^2}{t_i^2} + \frac{Delta_V^2}{V_i^2}}\right)} \quad (4.1)$$

Wobei hier der Mittelwert der Einzelfehler ($\Delta \frac{V_i}{t_i}$) quadratisch mit der Standardabweichung addiert wurde. Ich weiß nicht, ob das das richtige Vorgehen ist (gerne rückmelden, wie ich hier in Zukunft vorgehen soll). Den Fehler der Sinkzeiten habe ich am Anfang auch so berechnet. Dies ergibt für $\dot{V}$ einen sehr hohen relativen Fehler von: $\frac{\Delta \dot{V}}{\dot{V}} = 6\%$, der sich in den relativen Fehler $\frac{\Delta \eta}{\eta} = 6.5\%$ fortpflanzt.

Nimmt man nur die Standardabweichung als Fehler, so ergibt sich ein relativer Fehler von $\frac{\Delta \dot{V}}{\dot{V}} = 0.27\%$. Wenn man den Fitfehler einer Gerade durch die Messwerte legt, so erhält man denselben kleinen relativen Fehler durch die Kovarianzmatrix. Übrigens waren nach dem Fit noch 0.057 mL im Messbecher drinne, was mir sehr wenig vorkommt...

Nutzt man nun diesen kleineren Volumenstromfehler, ergibt sich $\eta^{\text{Hag.Pois.}} = (0.261 \pm 0.008) \text{ Pa s}$ und damit ein halb so kleiner relativer Fehler wie vorhin.

4.3. Vergleich der Versuchsteile

Die Abweichung der Messmethoden (siehe Ergebnis 3.2) zur Viskositätsbestimmung beläuft sich auf 0.4σ , ein sehr guter Wert. Nach Anpassen der Fehlerrechnung des Volumenstroms, indem nun der Fitfehler der Ausgleichsgeraden genutzt wurde, ergibt sich ein deutlich kleinerer Fehler:

$$\eta^{\text{Hag.-Pois.}} = (0.261 \pm 0.008) \text{ Pa s} \quad S = 0.8\sigma$$

Dadurch steigt die Sigma-Abweichung leicht, ist aber immer noch sehr gering.

Der Fehler nach Stokes ist überraschend klein, was wahrscheinlich am sehr geringen Fitfehler der dortigen linearen Gerade liegt.

Allgemein hätte ich erwartet, dass die Methode der Kapillarströmung genauer ist, weil hier keine menschliche Intervention geleistet werden muss (nur Ablesen der Volumenmarke). Ungenauigkeiten sind hier jedoch das Ablesen der Höhen, infolge dessen wird sowohl der Druck nicht exakt sein, als auch die Zeitmessung nicht immer genau zum Überschreiten der 5 mL-Marken erfolgt sein. Zudem haben wir vergessen, die Anfangshöhe zu messen, und diese dann nach Zurückschütten der aufgefangenen Strömungsflüssigkeit notiert. Und davor wurde auch mit dem Angelstab und Kugelbehälter ein kleiner Kugelregen erzeugt. Dies verfälscht natürlich die Höhenmessung.

4.4. Fazit

Eigentlich ein ganz netter Versuch. Aus irgendeinem Grund dauern Auswertungen bei mir aber immer ewig. Dennoch, war schön mal bisschen Fluidodynamik zu sehen. Jetzt hab ich richtig Lust, am Wasserhahn laminarflow einzustellen. Außerdem lieben wir Steve Mould: [The bizarre ripples that form in a stream of water](#). Der Stokes-Meme ist mir übrigens vor dem Einschlafen (kreativste Phase des Gehirns) selber eingefallen, aber natürlich kamen andere Menschen auf dieser Welt auch schon auf diese Idee.

Literaturquellen

¹Dr. J. Wagner, *Physikalisches Praktikum 2.1 für Studierende der Physik B.Sc.* [Online; Stand 03/2026] (Universität Heidelberg, März. 2026), https://git.physi.uni-heidelberg.de/schwierz/Versuchsanleitungen/src/branch/main/PAP2_1.pdf (besucht am 11.03.2026) (siehe S. 3–6, 18, 37).

²Dr. J. Wagner, *Physikalisches Praktikum 2.2 für Studierende der Physik B.Sc.* [Online; Stand 04/2026] (Universität Heidelberg, Apr. 2026), https://git.physi.uni-heidelberg.de/schwierz/Versuchsanleitungen/src/branch/main/PAP2_2.pdf (besucht am 14.04.2026) (siehe S. 9).

³Dimitrij Bathauer, *Zusatzaufgaben*, [Online; Stand 03/2026], <https://heibox.uni-heidelberg.de/d/dea242db205747a8b784/?p=%2F> (besucht am März. 2026) (siehe S. 15).

Abbildungsquellen

⁴Special meme fresh, *Stonks*, [Online; Stand 03/2026], <https://en.meming.world/wiki/Stonks> (siehe S. 1, 3); siehe *Porträt Sir Stokes*, <https://commons.wikimedia.org/wiki/File:Ggstokes.jpg>.

⁵Suvroc, *Definition of physical unit viscosity*, [Online; Stand 03/2026], https://commons.wikimedia.org/wiki/File:Definition_Viskositaet.png#/media/File:Definition_Viskositaet.png (siehe S. 3, 5).

⁶Netflix Series: *Narcos*, *Sad Pablo Escobar*, [Online; Stand 03/2026], https://en.meming.world/wiki/Pablo_Escobar_Waiting (siehe S. 3, 7).

A. Dichte von Polyethylenglykol

B. Memes

Vielen Dank für dein tolles PAP-Overkill-Dokument. Dein Humor und Support von Memes ist toll. Nachdem du damals auf meine Auswertung das Uni-Logo ein „Huh, warum ist das Uni-Logo?“ geeditet hast, comitte ich nun zu Intro-Memes.

Falls du noch Interesse an anderen Memes hast, ich hab hier noch eine Auswertung aus PAP1, auf die ich ziemlich stolz bin. Unter [Hierunter](#) sind neben dieser Auswertung noch ein paar passable Memes. (Mein Favorit ist V41).

C. Code-Anhang

Bei den Funktionen habe ich teilweise Dinge von verschiedenen Altprotokollen zusammengesucht, und auf meine Bedürfnisse umgeschrieben. Die konkreten Berechnung sind dann eigenständig.

March 26, 2026

```
[131]: # Numpy für bessere Berechnungen
import numpy as np
from numpy import exp, sqrt, log, pi
from uncertainties import unumpy as unp

# Weiteres für bessere Rechnungen
import pylab as py

from decimal import Decimal, ROUND_CEILING, getcontext # Besonders für sig.
↳ Runden
import math

# Berechnungen und Plotting
from scipy import odr
import scipy.optimize
import scipy.constants as c
from scipy.optimize import curve_fit
from scipy.stats import chi2
from scipy.stats import poisson
from scipy.integrate import quad
from scipy.signal import find_peaks
from scipy.signal import argrelextrema, argrelemin, argrelemax
from scipy.special import factorial

%matplotlib widget
import matplotlib.pyplot as plt
import matplotlib.mlab as mlab
import matplotlib.transforms as transforms

# Zum Auslesen von Dateien und ähnlichem
import os
import os.path
from pathlib import Path

import pandas as pd # Auch wichtig für den Latex Export
from pytexit import py2tex
import csv
```

```

import re
from typing import List

# Besseres Funktionen handling
import sympy as sp
from sympy import separatevars

# Display und Output
from IPython.display import display, Math, Latex, HTML

# Grafiken in EU-Größen ausgeben
def figsize_cm(width_cm, height_cm):
    return (2*width_cm / 2.54, 2*height_cm / 2.54)
def comma_to_float(valstr):
    return float(valstr.replace(',', '.'))

```

```

[132]: currentversuch = "212"
base = Path.cwd()
Messwerte_path = Path(base.parent.parent.parent/"Messwerte"/currentversuch)
↳ # initializing paths
Ausgabe_path = Path(base/"Diagramme")
print(Messwerte_path)
print(Ausgabe_path)

```

c:\Users\samue\OneDrive\Dokumente\PAP\PAP2.1\Messwerte\212

c:\Users\samue\OneDrive\Dokumente\PAP\PAP2.1\Auswertungen\Code\212\Diagramme

Runden signifikanter Stellen

```

[133]: def round_sig_digs(val, errVal):
    """
        Funktion zur Rundung eines Fehlers und die Anpassung des Messwertes daran.
        ↳ Diese Funktion wurde etwas umständlicher
        geschrieben, da python mit Floats und Runden schnell in Probleme rennt.
        ↳ Daher musste hier mit dezimal gearbeitet werden.
        Zudem sollten besonders kleine und große Messwerte in der
        ↳ Dezimalschreibweise geschrieben werden, damit diese auch
        für Protokolle geeignet sind.

        Parameter
        -----
        **val** : float
            Messwert

        **errVal** : float
            Ungenauigkeit des Messwertes
    """

```

```

Return
-----
**value_rounded** : str
    Gerundeter Messwert

**error_rounded** : str
    Gerundete Ungenauigkeit des Messwertes

**res** : str
    "Messwert \pm Fehler"
"""

# Daten zu Dezimal wechseln, da Floats probleme machen
val = Decimal(str(val))
errVal = Decimal(str(errVal))

exp = int(math.floor(math.log10(float(errVal)))) # Die erste
↳signifikante Stellenposition wird anhand des errVal bestimmt

if round(float(errVal / (Decimal(10) ** exp))) < 3: # wenn 1,2,3
↳erste signifikante Stelle, dann kommt eine zweite Nachkommastellenposition
↳hinzu
    exp -= 1

scale = Decimal(10) ** exp

val_round = (val / scale).quantize(Decimal('1'), rounding=ROUND_CEILING) *
↳scale # mit scale wird das Komma so verschoben, dass alle
↳signifikanten Stellen vor dem Komma stehen, und dann alle Nachkommastellen
↳weggerundet werden
err_round = (errVal / scale).quantize(Decimal('1'), rounding=ROUND_CEILING)
↳* scale

res = f"\num{{{val_round}}({err_round})}"
return float(val_round), float(err_round), res

# wissenschaftliche Notation erstmal vernachlässigen (da häufig Einheiten
↳gewollt sind, die 10~/e Prefixe beinhalten)

```

Gauss'sche Fehlerfortpflanzung

```

[134]: def gff(function, errPronePar, latex=True):
    """
    Kann die Fehlerformel einer gegebenen Gleichung bestimmen.

    Parameters
    -----

```

```

**func** : sympy function
    Funktion dessen Fehler bestimmt werden soll.

**errPronePar** : Array
    Liste (Array) aller fehlerbehafteten Größen der Gleichung con sp.Symbols
    Diese Werte werden als x_sym, y_sym, z_sym etc. bezeichnet und sind
    ↪ungleich den Werten für x, y, z.
    Für die Werte wird daher die Bezeichnung x_val, y_val, z_val etc.
    ↪genutzt und für deren Fehler err_x, err_y, err_z etc.

Return
-----
**absolut_err** : sympy function
    Gibt die Fehlergleichung des absoluten Fehlers wieder.

**relativ_err** : sympy function
    Gibt die Fehlergleichung des relativen Fehlers wieder.

**errProneParamters** : array
    Liste aller Fehlerbehafteten Größen
    ""

error = 0
errProneParamaters = []

function = sp.sympify(function)
variables = errPronePar.split(",")
variables = sp.symbols(variables)

for variable in variables:
    Delta = sp.Symbol(f"Delta_{variable}")
    partial = sp.diff(function, variable) # Die Funktion
    ↪wird nach der fehlerbehafteten Variable abgeleitet
    term=sp.UnevaluatedExpr(partial*Delta)**2
    error = error + ((term)) # Fehler werden
    ↪quadratisch aufsummiert
    errProneParamaters.append((variable,Delta))

absolute_err=sp.simplify(sp.sqrt(error),rational = True)
relative_err=sp.simplify(sp.sqrt(error/function**2),rational = True)

if latex:
    #Prints out the Latex Code for the Functions
    print("Funktion:")
    display(Math(sp.latex(function,long_frac_ratio=2)))
    print(sp.latex(function))
    print("Absoluter Fehler:")

```



```

display(Math(sp.latex(absolute_err,long_frac_ratio=2)))
print(sp.latex(absolute_err))
print("Relativer Fehler:")
display(Math(sp.latex(relative_err,long_frac_ratio=2)))
print(sp.latex(relative_err))
return function,absolute_err, relative_err, errProneParamaters

```

Sigma-Abweichungen

```

[135]: def sigma_abweichung(p1, p2, err_p1 = 0, err_p2 = 0):
        """
        Funktion zum berechnen der Sigma-Abweichung von zwei Messwerten, oder einem
        ↪Messwert und einem Literaturwert.
        """
        if err_p1 == 0 and err_p2 == 0:
            raise ValueError("Für die Sigma-Abweichung muss mindestens ein Wert
            ↪fehlerbehaftet sein!")
        else:
            abweichung=Decimal(str(abs(p1 - p2)/(np.sqrt(err_p1 ** 2+err_p2 ** 2))))

            exp = int(math.floor(math.log10(float(abweichung))))
            if round(float(abweichung / (Decimal(10) ** exp))) < 3:           # wenn
            ↪1,2,3 erste signifikante Stelle, dann kommt eine zweite
            ↪Nachkommastellenposition hinzu
                exp -= 1
            scale = Decimal(10) ** exp
            abweichung_round = (abweichung / scale).quantize(Decimal('1'),
            ↪rounding=ROUND_CEILING) * scale
            res = f"\num{{{abweichung_round}}}"

            return float(abweichung_round),res

```

```

[136]: geschwindigkeiten=np.array([0.0301,0.0253,0.0201,0.0148,0.0115,0.0071,0.0044,0.
        ↪00192,0.00060])
        delta_geschwindigkeiten=np.array([0.0007,0.0007,0.0005,0.0006,0.0009,0.0008,0.
        ↪0005,0.00020,0.00016])
        radien=np.array([9,8,7,6,5,4,3,2,1])*1e-3/2
        delta_r=0.025*1e-3/2
        R=75*1e-3/2

        plt.close('all')
        fig, ax = plt.subplots(figsize=figsize_cm(12,6))

        ax.set_title('')
        ax.set_ylabel('Sinkgeschwindigkeiten [m/s]')
        ax.set_xlabel(r'Kugelradien  $r^2$  [ $\mathrm{m}^2$ '])
        # ax.set_ylim(())

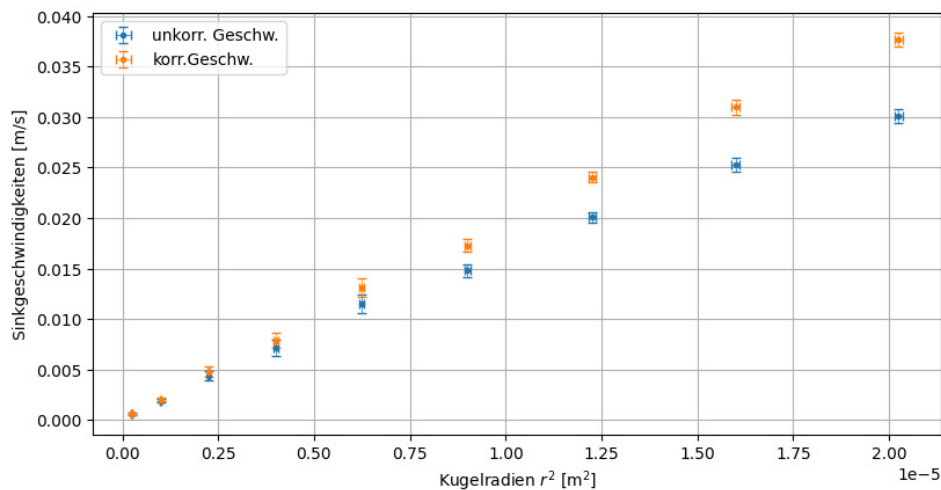
```

```

# ax.set_xlim(())
ax.
    ↳errorbar(x=radien**2,y=geschwindigkeiten,xerr=2*radien*delta_r,yerr=delta_geschwindigkeiten
    ↳",capsize=3,elinewidth=0.5,label='unkorr. Geschw. ')
ax.errorbar(x=radien**2,y=geschwindigkeiten*(1+2.1*radien/
    ↳R),xerr=2*radien*delta_r,yerr=delta_geschwindigkeiten,fmt="."
    ↳",capsize=3,elinewidth=0.5,label='korr.Geschw. ')

ax.grid()
ax.legend()
plt.show()
plt.savefig(Ausgabe_path/"Nr.1_Geschwindigkeiten.png",format="png")

```



```

[137]: # komplett useless berechnung, der Fehler ist so klein
Berechnung=gff("v*(1+2.1*r/R)", "v,r",False)[0]
Fehler_Berechnung=gff("v*(1+2.1*r/R)", "v,r",False)[1]
ergebnisse=[]
delta_ergebnisse=[]

for i in range(len(radien)):
    v=geschwindigkeiten[i]
    Delta_v=delta_geschwindigkeiten[i]
    r=radien[i]
    Delta_r=delta_r
    delta_ergebnisse.append(eval(str(Fehler_Berechnung)))
    ergebnisse.
    ↳append(round_sig_digs(eval(str(Berechnung)),delta_ergebnisse[i])[0])
    delta_ergebnisse[i]=round_sig_digs(ergebnisse[i],delta_ergebnisse[i])[1]

```

```

ergebnisse=np.array(ergebnisse)
delta_ergebniss=np.array(delta_ergebnisse)
print(ergebnisse)
print(delta_ergebnisse)
print(delta_geschwindigkeiten)

plt.close('all')
fig, ax = plt.subplots(figsize=figsize_cm(12,6))
x = np.linspace(0,2.1*1e-5,100)
ax.set_title('')
ax.set_ylabel('Sinkgeschwindigkeiten [m/s]')
ax.set_xlabel(r'Kugelradien  $r^2$  [ $\mathrm{m}^2$ ]')
# ax.set_ylim(())
# ax.set_xlim(())
ax.
    ↪errorbar(x=radien**2,y=geschwindigkeiten,xerr=2*radien*delta_r,yerr=delta_geschwindigkeiten
    ↪",capsize=3,elinewidth=0.5,label='unkorr. Geschw. ')
ax.
    ↪errorbar(x=radien**2,y=ergebnisse,xerr=2*radien*delta_r,yerr=delta_ergebnisse,fmt="
    ↪",capsize=3,elinewidth=0.5,label='korr.Geschw. ')

def lin(x,a):
    return (a*x)
popt, pcov = curve_fit(lin, radien**2,ergebnisse, sigma =_
    ↪delta_ergebnisse,absolute_sigma = True)
ax.errorbar(x, lin(x,*popt), label = "fit", color = "green")

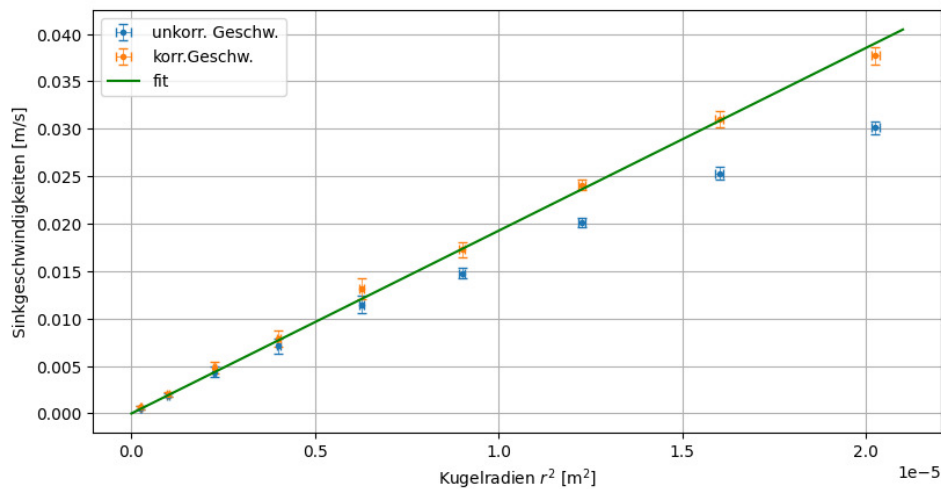
ax.grid()
ax.legend()
plt.show()
plt.savefig(Ausgabe_path/"Nr.1_Fit.png",format="png")
print(popt[0],np.sqrt(pcov[0][0]))

```

```

[0.0377  0.031   0.0241  0.0173  0.0132  0.0079  0.0048  0.00203 0.00062]
[0.0009, 0.0009, 0.0006, 0.0008, 0.0011, 0.0009, 0.0006, 0.00022, 0.00017]
[0.0007  0.0007  0.0005  0.0006  0.0009  0.0008  0.0005  0.0002  0.00016]

```



1926.7486784066791 26.219508117516206

```
[138]: rho_K = 1.385*1e-3/1e-6
Delta_rho_K = 0.0015*1e-3/1e-6
rho_F = 1.1478*1e-3/1e-6
Delta_rho_F = 0.0005*1e-3/1e-6
L_kap = 100e-3
Delta_L_kap = 0.5e-3

g=9.80984
Delta_g=0.00002
a=popt[0]
Delta_a=np.sqrt(pcov[0][0])

eta = eval(str(gff("2 * (rho_K - rho_F) * g / (9 * L_kap - 2 * rho_F * g^2 + 2 * rho_K * g^2)", "rho_K, rho_F, a, g", True)[0]))
Delta_eta = eval(str(gff("2 * (rho_K - rho_F) * g / (9 * L_kap - 2 * rho_F * g^2 + 2 * rho_K * g^2)", "rho_K, rho_F, a, g", False)[1]))
print(round_sig_digs(eta, Delta_eta)[2])
```

Funktion:

$$\frac{g}{9a} (-2\rho_F + 2\rho_K)$$

$$\frac{g}{9a} (-2\rho_F + 2\rho_K)$$

Absoluter Fehler:

$$\frac{2}{9} \sqrt{\frac{1}{a^4} (\Delta_a^2 g^2 (\rho_F - \rho_K)^2 + a^2 (\Delta_g^2 (-\rho_F + \rho_K)^2 + \Delta_{\rho_F}^2 g^2 + \Delta_{\rho_K}^2 g^2))}$$

$$\frac{2 \sqrt{\frac{\Delta_a^2}{a^2} g^2 \left(\rho_F - \rho_K \right)^2 + \Delta_g^2 \left(-\rho_F + \rho_K \right)^2 + \Delta_{\rho F}^2 g^2 + \Delta_{\rho K}^2 g^2}}{a^2 + \frac{\Delta_g^2 (-\rho_F + \rho_K)^2}{g^2 (\rho_F - \rho_K)^2} + \frac{\Delta_{\rho F}^2}{(\rho_F - \rho_K)^2} + \frac{\Delta_{\rho K}^2}{(\rho_F - \rho_K)^2}}$$

Relativer Fehler:

$$\sqrt{\frac{\Delta_a^2}{a^2} + \frac{\Delta_g^2 (-\rho_F + \rho_K)^2}{g^2 (\rho_F - \rho_K)^2} + \frac{\Delta_{\rho F}^2}{(\rho_F - \rho_K)^2} + \frac{\Delta_{\rho K}^2}{(\rho_F - \rho_K)^2}}$$

$$\sqrt{\frac{\Delta_a^2}{a^2} + \frac{\Delta_g^2 \left(-\rho_F + \rho_K \right)^2}{g^2 \left(\rho_F - \rho_K \right)^2} + \frac{\Delta_{\rho F}^2 g^2}{\left(\rho_F - \rho_K \right)^2} + \frac{\Delta_{\rho K}^2 g^2}{\left(\rho_F - \rho_K \right)^2}}$$

$$\text{num}\{0.269(0.005)\}$$

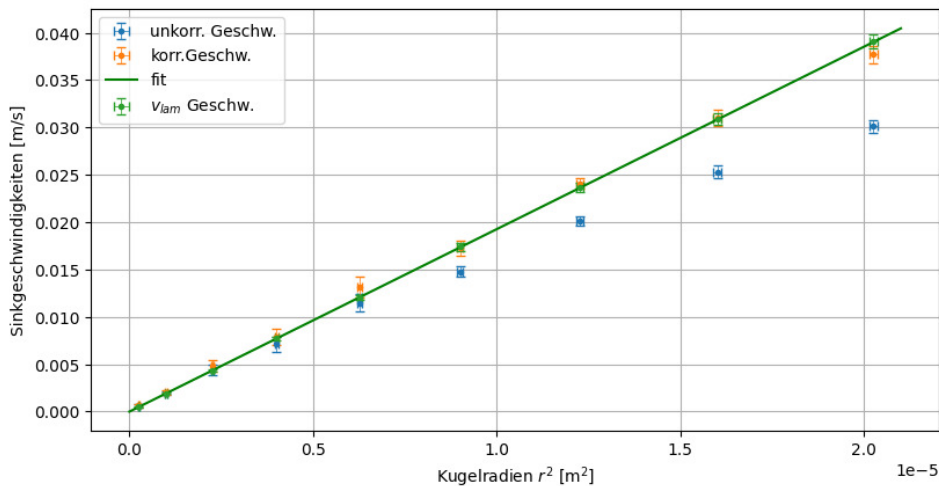
```
[139]: v_lam = 2 * g * (rho_K - rho_F) * radien**2 / (9*eta)
Delta_v_lam = v_lam * np.sqrt((Delta_g/g)**2 + (2 * Delta_r/radien)**2 +
    ↪ (Delta_eta/eta)**2 + (Delta_rho_F/(rho_K - rho_F))**2 + (Delta_rho_K/(rho_K -
    ↪ rho_F))**2)
for i in range(len(v_lam)):
    v_lam[i]=round_sig_digs(v_lam[i],Delta_v_lam[i])[0]
    Delta_v_lam[i]=round_sig_digs(v_lam[i],Delta_v_lam[i])[1]
    ↪
    ↪ print(f"\num{{{radien[i]*1e3}}}&\num{{{ergebnisse[i]}({delta_ergebniss[i]})}}&{round_sig_
print(v_lam,Delta_v_lam)

plt.close('all')
fig, ax = plt.subplots(figsize=figsize_cm(12,6))
x = np.linspace(0,2.1*1e-5,100)
ax.set_title('')
ax.set_ylabel('Sinkgeschwindigkeiten [m/s]')
ax.set_xlabel(r'Kugelradien $r^2$ [$\mathrm{m}^2$]')
# ax.set_ylim(())
# ax.set_xlim(())
ax.
    ↪ errorbar(x=radien**2,y=geschwindigkeiten,xerr=2*radien*delta_r,yerr=delta_geschwindigkeiten
    ↪ ",capsize=3,elinewidth=0.5,label='unkorr. Geschw.')
ax.
    ↪ errorbar(x=radien**2,y=ergebnisse,xerr=2*radien*delta_r,yerr=delta_ergebnisse,fmt="
    ↪ ",capsize=3,elinewidth=0.5,label='korr.Geschw.')

def lin(x,a):
    return (a*x)
popt, pcov = curve_fit(lin, radien**2,ergebnisse, sigma =
    ↪ delta_ergebnisse,absolute_sigma = True)
ax.errorbar(x, lin(x,*popt), label = "fit", color = "green")
ax.errorbar(x=radien**2,y=v_lam,xerr=2*radien*delta_r,yerr=Delta_v_lam,fmt="
    ↪ ",capsize=3,elinewidth=0.5,label='$v_{lam}$ Geschw.')
```

```
ax.grid()
ax.legend()
plt.show()
plt.savefig(Ausgabe_path/"Nr.1_Fitmitlam.png",format="png")
```

```
\num{4.500000000000001}&\num{0.0377(0.0009)}&\num{0.0391(0.0007)}\\
\num{4.0}&\num{0.031(0.0009)}&\num{0.0309(0.0006)}\\
\num{3.5}&\num{0.0241(0.0006)}&\num{0.0237(0.0005)}\\
\num{3.0}&\num{0.0173(0.0008)}&\num{0.0174(0.0004)}\\
\num{2.5}&\num{0.0132(0.0011)}&\num{0.01205(0.00024)}\\
\num{2.0}&\num{0.0079(0.0009)}&\num{0.00771(0.00016)}\\
\num{1.5}&\num{0.0048(0.0006)}&\num{0.00434(0.00011)}\\
\num{1.0}&\num{0.00203(0.00022)}&\num{0.00193(0.00006)}\\
\num{0.5}&\num{0.00062(0.00017)}&\num{0.00049(0.00003)}\\
[0.0391 0.0309 0.0237 0.0174 0.01205 0.00771 0.00434 0.00193 0.00049]
[7.0e-04 6.0e-04 5.0e-04 4.0e-04 2.4e-04 1.6e-04 1.1e-04 6.0e-05 3.0e-05]
```



```
[140]: gff("2 * g * (rho_K - rho_F) * r**2 / (9*eta)", "g,rho_K,rho_F,r,eta", True)
```

Funktion:

$$\frac{2gr^2}{9\eta}(-\rho_F + \rho_K)$$

$\frac{2}{9} g r^2 (-\rho_F + \rho_K)$

Absoluter Fehler:

$$\frac{2}{9} \sqrt{\frac{r^2}{\eta^4} \left(\Delta_\eta^2 g^2 r^2 (\rho_F - \rho_K)^2 + \eta^2 \left(\Delta_g^2 r^2 (-\rho_F + \rho_K)^2 + 4\Delta_r^2 g^2 (-\rho_F + \rho_K)^2 + \Delta_{\rho_F}^2 g^2 r^2 + \Delta_{\rho_K}^2 g^2 r^2 \right) \right)}$$

$$\frac{2 \sqrt{\frac{r^2}{\left(\Delta_{\eta}^2 g^2 r^2 \left(\rho_F - \rho_K\right)^2 + \eta^2 \left(\Delta_g^2 r^2 \left(-\rho_F + \rho_K\right)^2 + 4 \Delta_r^2 g^2 \left(-\rho_F + \rho_K\right)^2 + \Delta_{\rho F}^2 g^2 r^2 + \Delta_{\rho K}^2 g^2 r^2\right)\right)}}{\eta^2} + \frac{\Delta_g^2 (-\rho_F + \rho_K)^2}{g^2 (\rho_F - \rho_K)^2} + \frac{4 \Delta_r^2 (-\rho_F + \rho_K)^2}{r^2 (\rho_F - \rho_K)^2} + \frac{\Delta_{\rho F}^2}{(\rho_F - \rho_K)^2} + \frac{\Delta_{\rho K}^2}{(\rho_F - \rho_K)^2}$$

Relativer Fehler:

$$\sqrt{\frac{\Delta_{\eta}^2}{\eta^2} + \frac{\Delta_g^2 (-\rho_F + \rho_K)^2}{g^2 (\rho_F - \rho_K)^2} + \frac{4 \Delta_r^2 (-\rho_F + \rho_K)^2}{r^2 (\rho_F - \rho_K)^2} + \frac{\Delta_{\rho F}^2}{(\rho_F - \rho_K)^2} + \frac{\Delta_{\rho K}^2}{(\rho_F - \rho_K)^2}}$$

$$\sqrt{\frac{\Delta_{\eta}^2}{\eta^2} + \frac{\Delta_g^2}{g^2} \left(\frac{-\rho_F + \rho_K}{\rho_F - \rho_K}\right)^2 + \frac{\Delta_r^2}{r^2} \left(\frac{-\rho_F + \rho_K}{\rho_F - \rho_K}\right)^2 + \frac{\Delta_{\rho F}^2}{(\rho_F - \rho_K)^2} + \frac{\Delta_{\rho K}^2}{(\rho_F - \rho_K)^2}}$$

[140]: $(2 * g * r^2 * (-\rho_F + \rho_K) / (9 * \eta),$
 $2 * \sqrt{r^2 * (\Delta_{\eta}^2 * g^2 * r^2 * (\rho_F - \rho_K)^2 +$
 $\eta^2 * (\Delta_g^2 * r^2 * (-\rho_F + \rho_K)^2 + 4 * \Delta_r^2 * g^2 * (-\rho_F +$
 $\rho_K)^2 + \Delta_{\rho F}^2 * g^2 * r^2 + \Delta_{\rho K}^2 * g^2 * r^2)) / \eta^4} / 9,$
 $\sqrt{(\Delta_{\eta}^2 / \eta^2 + \Delta_g^2 * (-\rho_F + \rho_K)^2 / (g^2 * (\rho_F -$
 $\rho_K)^2) + 4 * \Delta_r^2 * (-\rho_F + \rho_K)^2 / (r^2 * (\rho_F - \rho_K)^2) +$
 $\Delta_{\rho F}^2 / (\rho_F - \rho_K)^2 + \Delta_{\rho K}^2 / (\rho_F - \rho_K)^2)},$
 $[(g, \Delta_g),$
 $(\rho_K, \Delta_{\rho K}),$
 $(\rho_F, \Delta_{\rho F}),$
 $(r, \Delta_r),$
 $(\eta, \Delta_{\eta})]$

```
[141]: Re = rho_F * geschwindigkeiten * 2 * radien / eta
Delta_Re = Re * np.sqrt((Delta_r / radien)**2 + (Delta_eta / eta)**2 + (Delta_rho_F /
    rho_F)**2 + (delta_geschwindigkeiten / geschwindigkeiten)**2)
for i in range(len(Re)):
    Re[i] = round_sig_digs(Re[i], Delta_Re[i])[0]
    Delta_Re[i] = round_sig_digs(Re[i], Delta_Re[i])[1]
print(Re, Delta_Re)
```

```
[1.16  0.87  0.602  0.38  0.246  0.122  0.057  0.0165  0.0026] [0.04  0.03
0.018  0.017  0.02  0.014  0.007  0.0018  0.0007]
```

```
[142]: gff("rho_F * v * 2 * r / eta", "rho_F, v, r, eta", True)
```

Funktion:

$$\frac{2r}{\eta} \rho_F v$$

$$\frac{2 r \rho_F v}{\eta}$$

Absoluter Fehler:

$$2\sqrt{\frac{1}{\eta^4} \left(\Delta_\eta^2 r^2 \rho_F^2 v^2 + \eta^2 \left(\Delta_r^2 \rho_F^2 v^2 + \Delta_{\rho_F}^2 r^2 v^2 + \Delta_v^2 r^2 \rho_F^2 \right) \right)}$$

$$2 \sqrt{\frac{\Delta_\eta^2 r^2 \rho_F^2 v^2 + \eta^2 \left(\Delta_r^2 \rho_F^2 v^2 + \Delta_{\rho_F}^2 r^2 v^2 + \Delta_v^2 r^2 \rho_F^2 \right)}{\eta^4}}$$

Relativer Fehler:

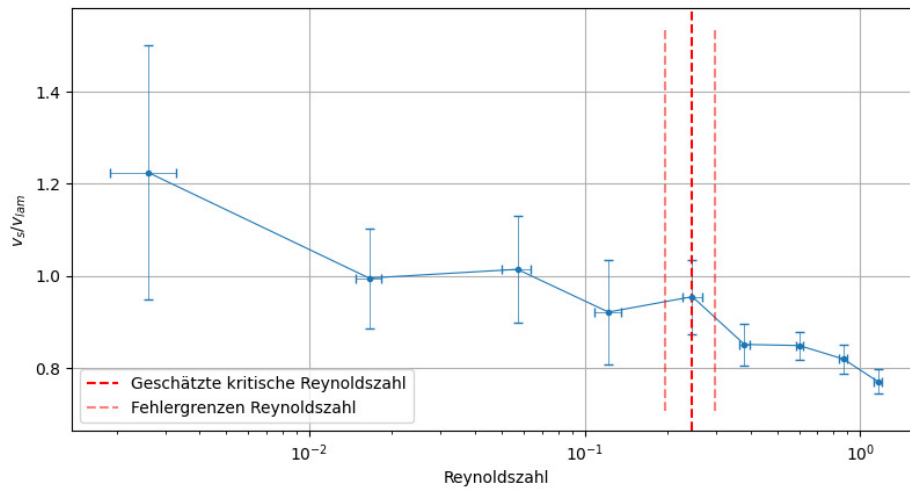
$$\sqrt{\frac{\Delta_\eta^2}{\eta^2} + \frac{\Delta_r^2}{r^2} + \frac{\Delta_{\rho_F}^2}{\rho_F^2} + \frac{\Delta_v^2}{v^2}}$$

$$\sqrt{\frac{\Delta_\eta^2 r^2 \rho_F^2 v^2}{\eta^4} + \frac{\Delta_r^2 \rho_F^2 v^2}{r^2} + \frac{\Delta_{\rho_F}^2 r^2 v^2}{\rho_F^2} + \frac{\Delta_v^2 r^2 \rho_F^2}{v^2}}$$

```
[142]: (2*r*rho_F*v/eta,
        2*sqrt((Delta_eta**2*r**2*rho_F**2*v**2 + eta**2*(Delta_r**2*rho_F**2*v**2 +
        Delta_rho_F**2*r**2*v**2 + Delta_v**2*r**2*rho_F**2))/eta**4),
        sqrt(Delta_eta**2/eta**2 + Delta_r**2/r**2 + Delta_rho_F**2/rho_F**2 +
        Delta_v**2/v**2),
        [(rho_F, Delta_rho_F), (v, Delta_v), (r, Delta_r), (eta, Delta_eta)])
```

```
[143]: vs_vlam = geschwindigkeiten/v_lam
Delta_vs_vlam = np.sqrt((delta_geschwindigkeiten/geschwindigkeiten)**2 +
    ↪(geschwindigkeiten * Delta_v_lam/v_lam**2)**2)

plt.close('all')
fig, ax = plt.subplots(figsize=figsize_cm(12,6))
ax.set_xscale('log')
ax.set_ylabel('$v_s/v_{\text{lam}}$')
ax.set_xlabel('Reynoldszahl')
# ax.set_ylim(())
# ax.set_xlim(())
ax.errorbar(x=Re,y=vs_vlam,xerr=Delta_Re,yerr=Delta_vs_vlam,fmt=".",
    ↪",capsize=3,elinewidth=0.5)
ax.plot(Re,vs_vlam,linestyle="--",color="C0",linewidth=0.8)
ax.axvline(0.246, color = "red", linestyle = "dashed", label = "Geschätzte_
    ↪kritische Reynoldszahl")
ymin,ymax=ax.get_ylim()
ax.vlines(x=[0.196,0.296],ymin=ymin,ymax=ymax,color = "red", linestyle =
    ↪"dashed", alpha=0.5, label = "Fehlergrenzen Reynoldszahl")
ax.grid()
ax.legend()
plt.show()
plt.savefig(Ausgabe_path/"Nr.1_Reynoldszahl.png",format="png")
```

```
[144]: #Aufgabe 2 Daten
r_kap = 0.75e-3
Delta_r_kap = 0.005e-3
L_kap = 100e-3
Delta_L_kap = 0.5e-3
rho_F=1.1474*1e-3/1e-6
Delta_rho_F=0.0006*1e-3/1e-6
V = np.linspace(10,30,5)* 1e-6
Delta_V=1e-6
T = np.array([344.2,514.4,689.8,862.5,1033.8])
Delta_T = 1
h = 540*1e-3
Delta_h = 0.7*1e-3

p = rho_F * g * h
Delta_p = p*np.sqrt((Delta_h/h)**2 + (Delta_rho_F/rho_F)**2 + (Delta_g/g)**2)
print(round_sig_digs(p,Delta_p)[0],round_sig_digs(p,Delta_p)[1])
print(p,Delta_p)
dVdt=np.mean(V/T)
Delta_dVdt=np.sqrt((np.std(V/T))**2
                    # +np.mean((V/T)*np.sqrt(Delta_T**2/T**2 + Delta_V**2/
                    ↪ V**2))**2)
print(round_sig_digs(dVdt,Delta_dVdt)[0],round_sig_digs(dVdt,Delta_dVdt)[1])
eta2 = np.pi * p * r_kap**4 / (8*dVdt*L_kap)
Delta_eta2 = eta2 * np.sqrt((Delta_p/p)**2 + (4*Delta_r_kap/r_kap)**2 +
                             ↪ (Delta_dVdt/dVdt)**2 + (Delta_L_kap/L_kap)**2)
print(round_sig_digs(eta2,Delta_eta2)[0],round_sig_digs(eta2,Delta_eta2)[1])
```

6079.0 9.0

```
6078.1376246400005 8.495999425292306
2.905e-08 7e-11
0.261 0.008
```

```
[145]: def linm(x, m, b):
        return m * x + b
popt, pcov = curve_fit(linm, T, V)
print(round_sig_digs(popt[0], np.sqrt(pcov[0][0]))[0], round_sig_digs(popt[0],
↪ np.sqrt(pcov[0][0]))[1])
a=round_sig_digs(popt[0], np.sqrt(pcov[0][0]))[0]
delta_a=round_sig_digs(popt[0], np.sqrt(pcov[0][0]))[1]

eta2 = np.pi * p * r_kap**4 / (8*dVdt*L_kap)
Delta_eta2 = eta2 * np.sqrt((Delta_p/p)**2 + (4*Delta_r_kap/r_kap)**2 +
↪ (delta_a/a)**2 + (Delta_L_kap/L_kap)**2)
print(round_sig_digs(eta2, Delta_eta2)[0], round_sig_digs(eta2, Delta_eta2)[1])
```

```
2.895e-08 8e-11
0.261 0.008
```

```
[146]: Re2 = 2 * rho_F/eta2 * dVdt/(np.pi * r_kap)
Delta_Re2 = Re2 * np.sqrt(((Delta_dVdt)/dVdt)**2 + (Delta_rho_F/rho_F)**2 +
↪ (Delta_eta2/eta2)**2 + (Delta_r_kap/r_kap)**2)
print(round_sig_digs(Re2, Delta_Re2)[2])
```

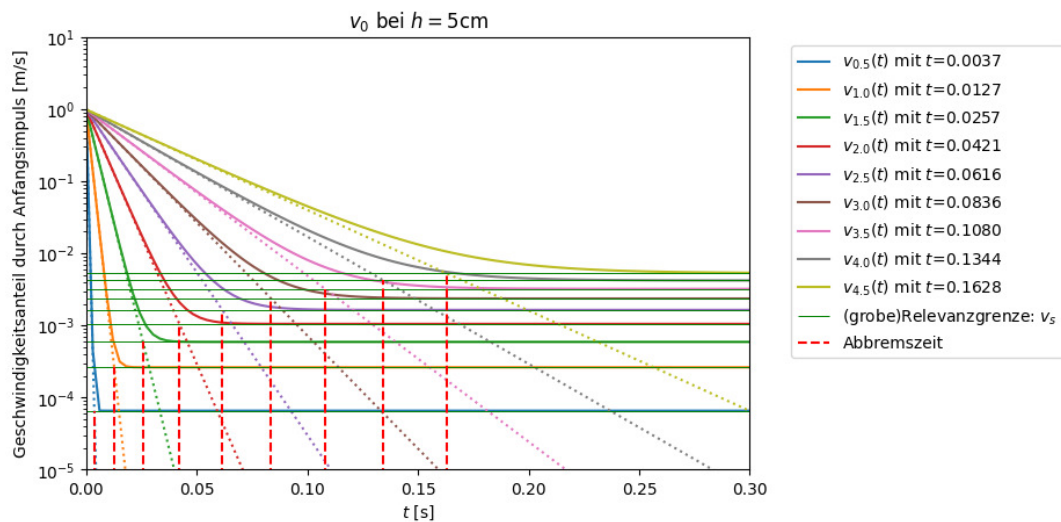
```
\num{0.109(0.004)}
```

ZUSATZ WENIGSTENS NOCHMAL NE INTERESSANTE SACHE

```
[147]: plt.close()
plt.figure(figsize=figsize_cm(12,6))
t = np.linspace(0,0.3,100) #Wir schauen uns Zeiten von 0 bis 1s in 100er
↪ Schritten an und plotten das.
r = np.array([1,2,3,4,5,6,7,8,9])*1e-3/2
a = 6 * c.pi * 0.2 * r / (4/3*c.pi*1.385e3*r**3)
v_s = 2/9*(1.385-1.148)*1e3*r**2/0.2
v_0=np.sqrt(2*9.81*0.05)
t_schnitt = np.log(v_s / v_0) / -a
def velocity(t,a,v,v_s):
    return v_s*(1-exp(-a*t))+v * np.exp(-a*t)
def velocity2(t,a,v):
    return v*np.exp(-a*t)
for i in range(len(r)):
    plt.plot(t,velocity(t,a[i],v_0,v_s[i]), label = f"$v_{\{r[i]*1e3:.1f\}}(t)$",
↪ mit $t$={t_schnitt[i]:.4f})
    plt.plot(t,velocity2(t,a[i],v_0), color=f"C{i}",linestyle = "dotted")

plt.hlines(y = v_s, xmin = 0, xmax = 2, color = 'green', linewidth=0.7,label =
↪ '(grobe)Relevanzgrenze: $v_s$')
```

```
plt.vlines(x=t_schnitt,ymin=1e-20,ymax=v_s, color='red', linestyle='dashed',
          label=f'Abbremszeit')
plt.yscale('log')
plt.ylim((1e-5,1e1))
plt.xlim((0,0.3))
plt.xlabel('$t$ [s]')
plt.ylabel('Geschwindigkeitsanteil durch Anfangsimpuls [m/s]')
plt.title('$v_0$ bei $h=5$cm')
plt.legend(bbox_to_anchor=(1.05, 1),loc='upper left')
plt.tight_layout()
plt.show()
plt.savefig(Ausgabe_path/"Zusatzreal.png",format="png")
```



[148]: `gff("V/T", "V,T", True)`

Funktion:

$$\frac{V}{T}$$

$\frac{V}{T}$

Absoluter Fehler:

$$\sqrt{\frac{1}{T^4} (\Delta_T^2 V^2 + \Delta_V^2 T^2)}$$

$$\sqrt{\frac{\Delta_T^2 V^2 + \Delta_V^2 T^2}{T^4}}$$

Relativer Fehler:

$$\sqrt{\frac{\Delta_T^2}{T^2} + \frac{\Delta_V^2}{V^2}}$$

$$\sqrt{\frac{\Delta_T^2}{T^2}} + \frac{\Delta_V^2}{V^2}$$

[148]: (V/T,
sqrt((Delta_T**2*V**2 + Delta_V**2*T**2)/T**4),
sqrt(Delta_T**2/T**2 + Delta_V**2/V**2),
[(V, Delta_V), (T, Delta_T)])

[149]: gff("pi * p * r_kap**4 / (8*dVdt*L_kap)", "p,r_kap,dVdt,L_kap", True)

Funktion:

$$\frac{\pi p r_{kap}^4}{8 L_{kap} dV dt}$$

$$\frac{\pi p r_{kap}^4}{8 L_{kap} dV dt}$$

Absoluter Fehler:

$$\frac{\pi}{8} \sqrt{\frac{r_{kap}^6}{L_{kap}^4 dV dt^4} (\Delta_{L_{kap}}^2 dV dt^2 p^2 r_{kap}^2 + \Delta_{dV dt}^2 L_{kap}^2 p^2 r_{kap}^2 + L_{kap}^2 dV dt^2 (\Delta_p^2 r_{kap}^2 + 16 \Delta_{r_{kap}}^2 p^2))}$$

$$\frac{\pi}{8} \sqrt{\frac{r_{kap}^6}{L_{kap}^4 dV dt^4} (\Delta_{L_{kap}}^2 dV dt^2 p^2 r_{kap}^2 + \Delta_{dV dt}^2 L_{kap}^2 p^2 r_{kap}^2 + L_{kap}^2 dV dt^2 (\Delta_p^2 r_{kap}^2 + 16 \Delta_{r_{kap}}^2 p^2))}$$

Relativer Fehler:

$$\sqrt{\frac{\Delta_{L_{kap}}^2}{L_{kap}^2} + \frac{\Delta_{dV dt}^2}{dV dt^2} + \frac{\Delta_p^2}{p^2} + \frac{16}{r_{kap}^2} \Delta_{r_{kap}}^2}$$

$$\sqrt{\frac{\Delta_{L_{kap}}^2}{L_{kap}^2} + \frac{\Delta_{dV dt}^2}{dV dt^2} + \frac{\Delta_p^2}{p^2} + \frac{16}{r_{kap}^2} \Delta_{r_{kap}}^2}$$

[149]: (pi*p*r_kap**4/(8*L_kap*dVdt),
pi*sqrt(r_kap**6*(Delta_L_kap**2*dVdt**2*p**2*r_kap**2 +
Delta_dVdt**2*L_kap**2*p**2*r_kap**2 + L_kap**2*dVdt**2*(Delta_p**2*r_kap**2 +
16*Delta_r_kap**2*p**2))/(L_kap**4*dVdt**4))/8,
sqrt(Delta_L_kap**2/L_kap**2 + Delta_dVdt**2/dVdt**2 + Delta_p**2/p**2 +
16*Delta_r_kap**2/r_kap**2),
[(p, Delta_p),
(r_kap, Delta_r_kap),
(dVdt, Delta_dVdt),
(L_kap, Delta_L_kap)])

[150]: print(sigma_abweichung(0.268,0.261,0.018,0.004)[1])
print(sigma_abweichung(0.268,0.261,0.008,0.004)[1])

\num{0.4}
\num{0.8}

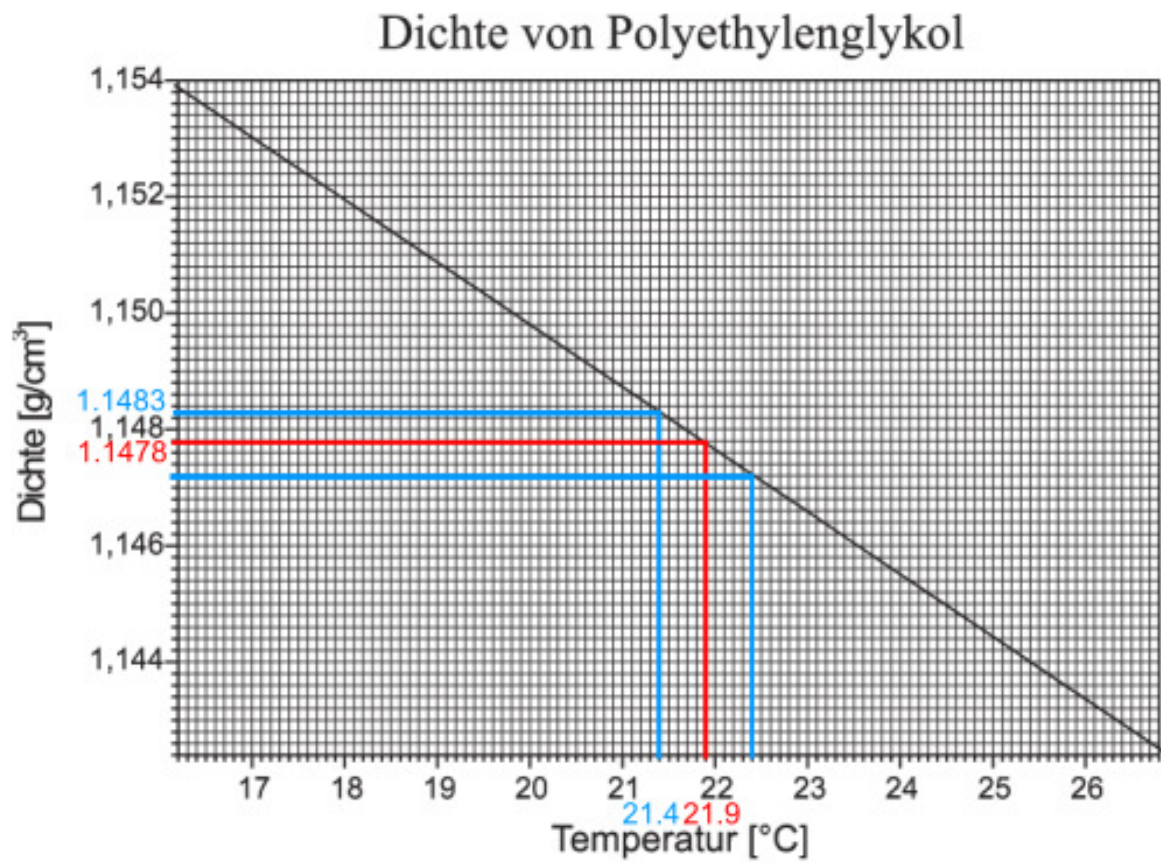


Abb. A.1: Dichte von Polyethylenglykol¹